CPSC 2150 Project Report

Yash Patel

Requirements Analysis

Functional Requirements:

1. As a player, I need to place a marker so that I can indicate my turn
2. As a player, I want to restart the game after drawing so I can continue playing.
3. As a player, I need to alternate players so that I can have each of them have a turn playing.
4. As a programmer, I need to have a computer to play with, so I can view and test my game.
5. As a player, I need to be able to place my marker anywhere on a 5 x 8 grid, so then I can use my turn.
6. As a gamescreen, I need to know when all the spaces are filled with no winning side so that it can display the draw message.
7. As a player, I want to be able to quit the game so that I can stop playing
8. As a player, I need to be player 'X' if I am first so that I can start the game
9. As a gameboard, I need to place either 'X' or 'O', so I know the game pieces
10. As a gameboard, I need to check to see if the row/col is valid so that I can place a marker there
11. As a gameboard, I need to know if 5 of the same markers are placed in the same column to count as a vertical win
12. As a gameboard, I need to know if 5 of the same markers are placed in the same row to count as a horizontal win
13. As a gameboard, I need to know if 5 of the same markers are placed in a diagonal line to count as a diagonal win

Non-Functional Requirements

1. The program is written in java
2. The program is played using the terminal
3. The program must have two players
4. The program must be developed in IntelliJ
5. The player must know the rules of tic-tac-toe

6. the program must have a 5 x 8 board
7. The ‘X’ Player must go first

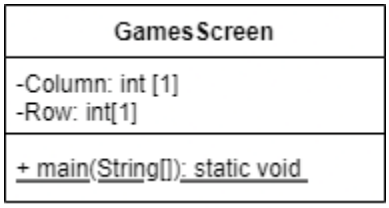
Deployment Instructions

Details in Projects 2-5.

System Design

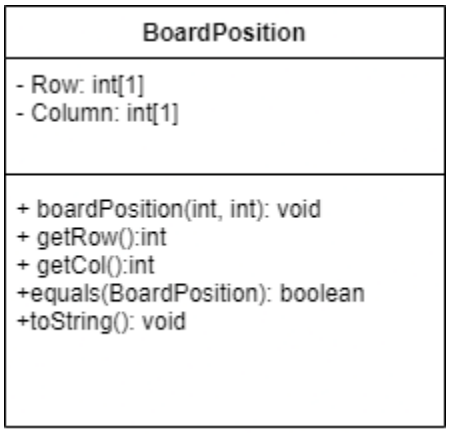
Class 1: GameScreen

Class diagram



Class 2: BoardPosition

Class diagram



Class 3: GameBoard

Class diagram

GameBoard
- PlayerChar: char[*] {'X', 'O'} - Board: char[*][*]
+ GameBoard(): void + checkSpace(BoardPosition): boolean + placeMarker(BoardPosition, char): void + checkForWinner(BoardPosition): boolean + checkForDraw(): boolean + checkHorizontalWin(BoardPosition, char): boolean + checkVerticalWin(BoardPosition, char): boolean + checkDiagonalWin(BoardPosition, char): boolean + whatsAtPos(BoardPosition): char + isPlayerAtPos(BoardPosition, char): boolean + toString():string

Class 4: IGameBoard

Class diagram

IGameBoard
-rowNum: int[1] -colNum: int[1] -winCount: int[1]
+ getNumRows(): int <default> + getNumColumns(): int <default> + getNumToWin(): int <default> + checkSpace(BoardPosition): boolean <default> + placeMarker(BoardPosition, char): void + checkForWinner(BoardPosition): boolean <default> + checkVerticalWin(BoardPosition): boolean <default> + checkHorizontalWin(BoardPosition): boolean <default> + checkDiagonalWin(BoardPosition): boolean <default> + checkForDraw(): boolean <default> + whatsAtPos(boardPosition): char + isPlayerAtPos(boardPosition, char): boolean <default>

Class 5: AbsGameBoard

Class diagram

AbsGameBoard
+ toString(): string

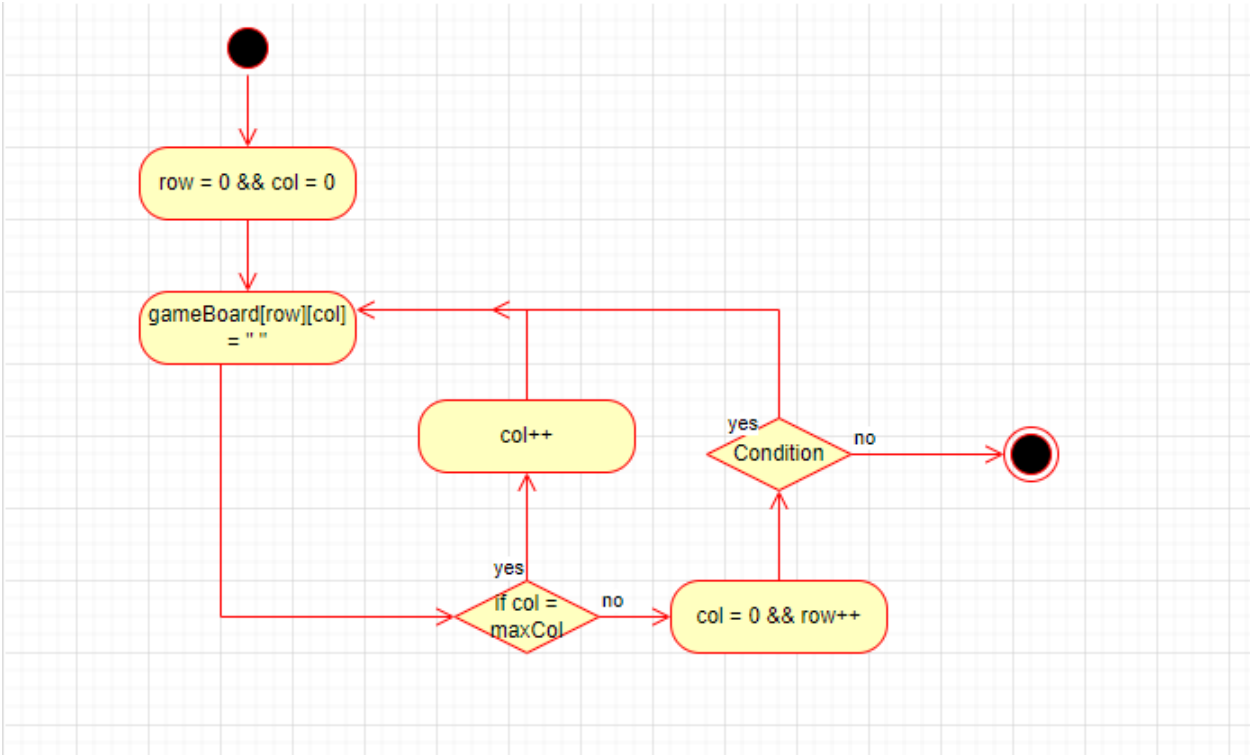
Class 6: GameBoardMem

Class diagram

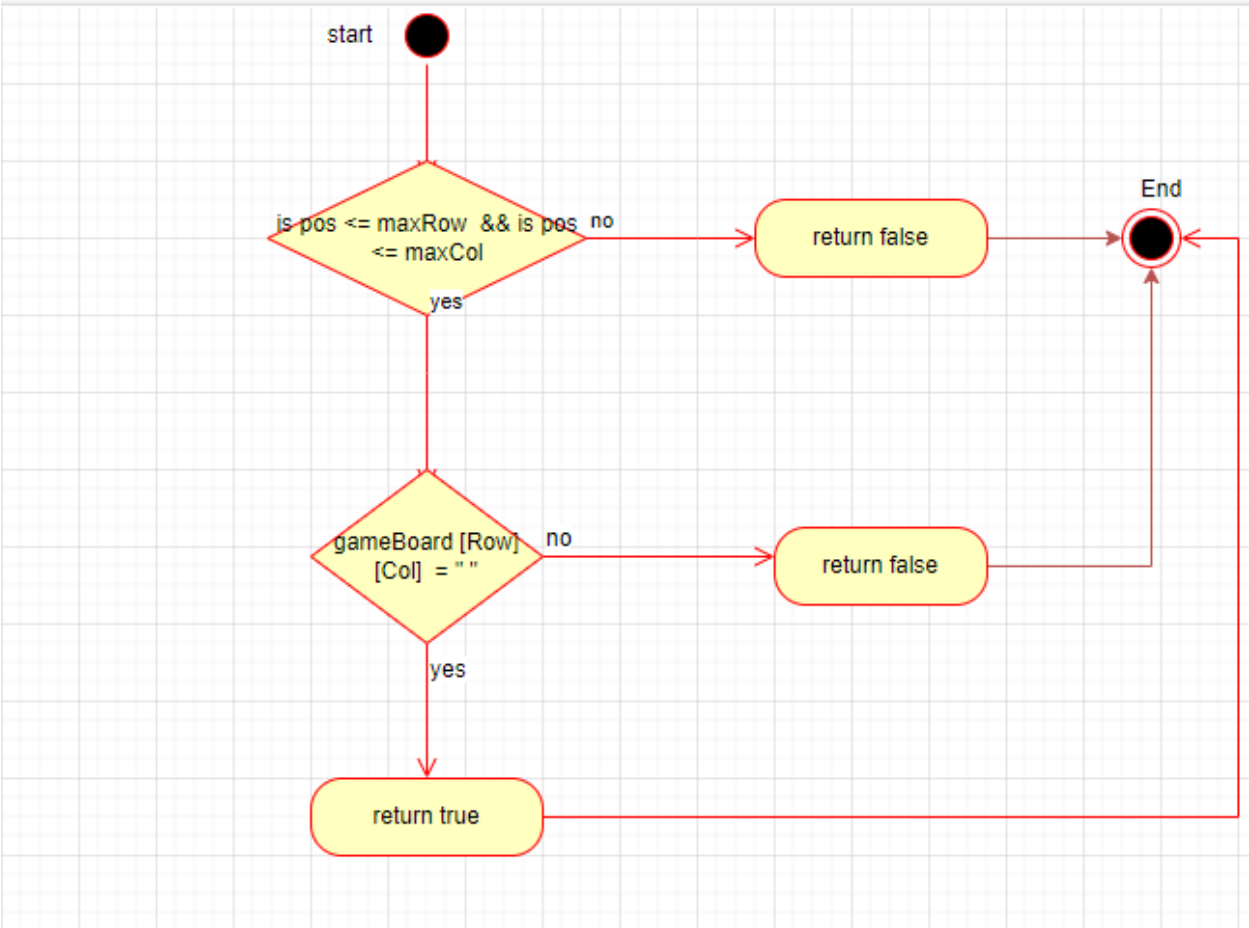
GameBoardMem
- Map<Character, List<BoardPosition>> board: board - playerChar: char - winScore: int - row: int - col: int
+ GameBoardMem(int,int,int) + placeMarker(BoardPosition,char): void + whatsAtPos(BoardPosition): char + isPlayerAtPos(BoardPosition, char): boolean + getNumRows(): int + getNumColumns(): int +getNumToWIn(): int

Activity diagrams

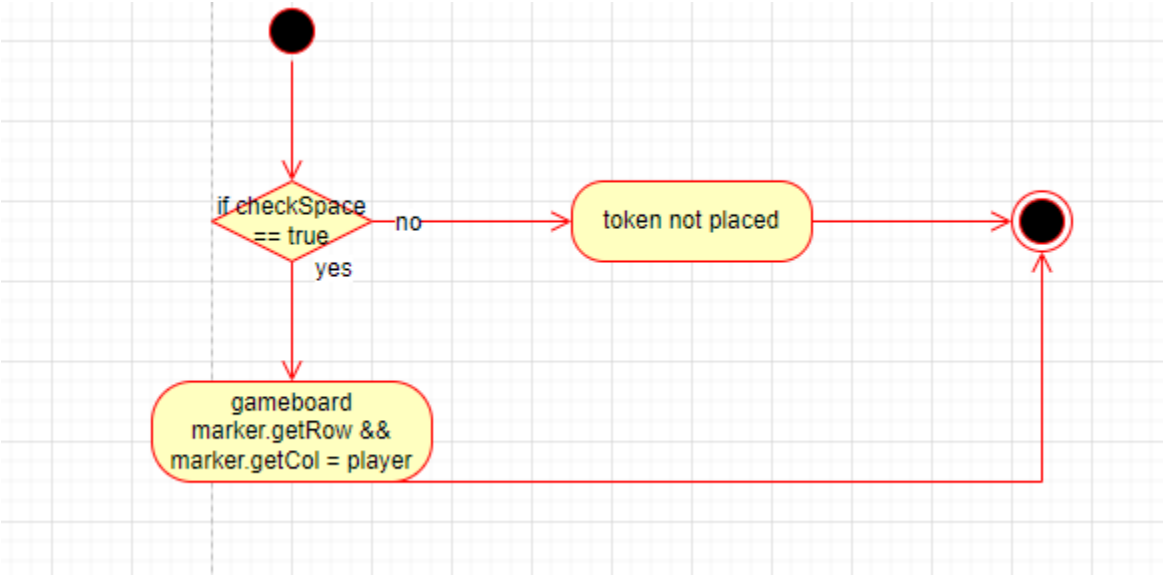
Gameboard(Constructor):



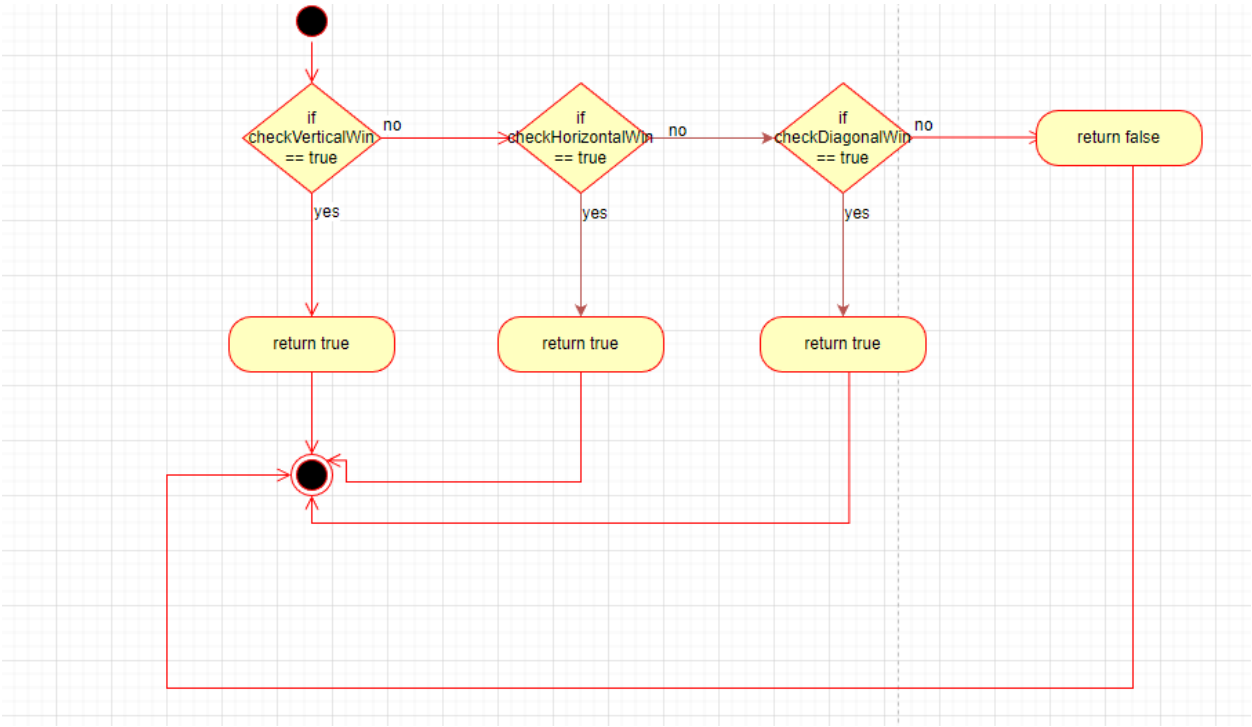
public boolean checkSpace(BoardPosition pos):



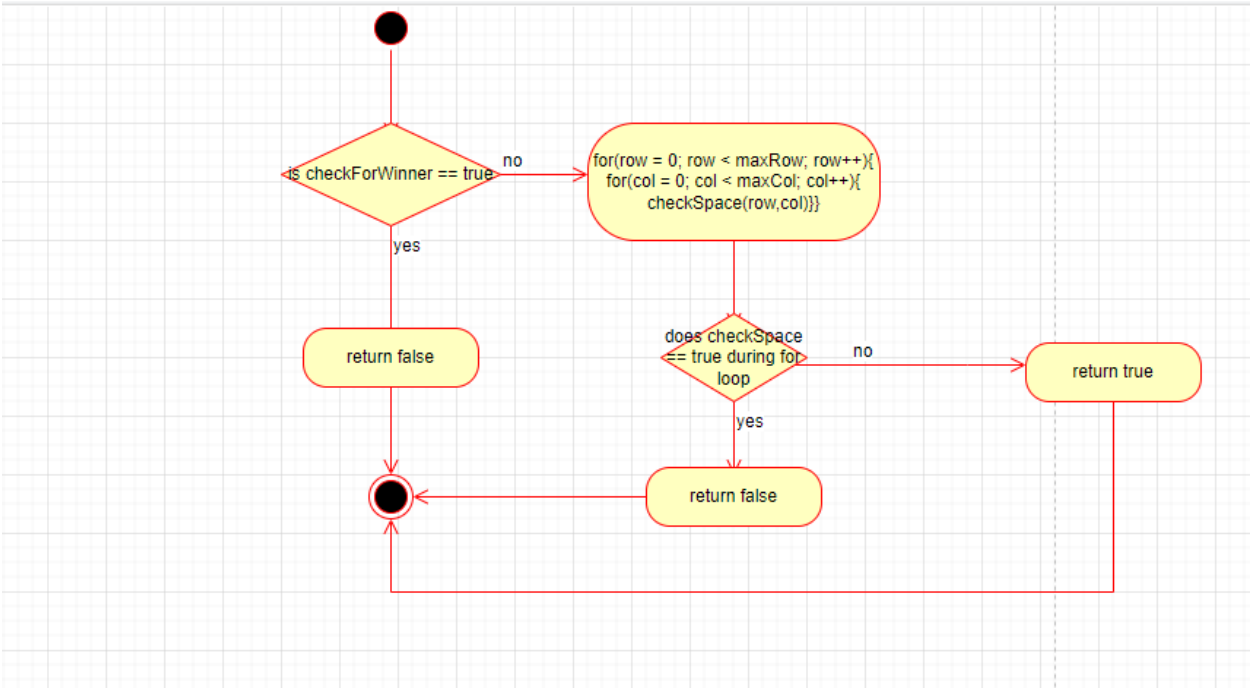
public void placeMarker(BoardPosition marker, char player):



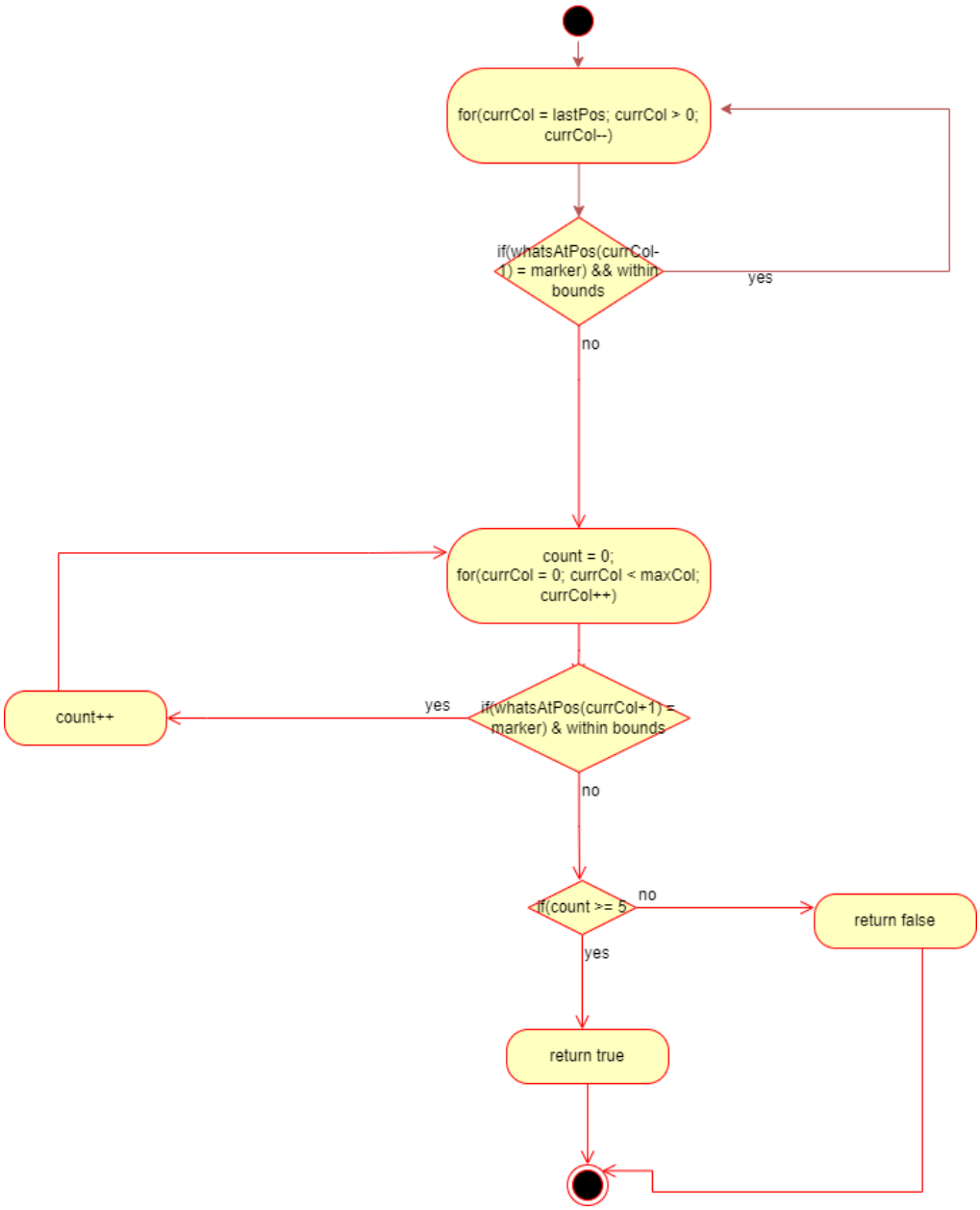
public boolean checkForWinner(BoardPosition lastPos):



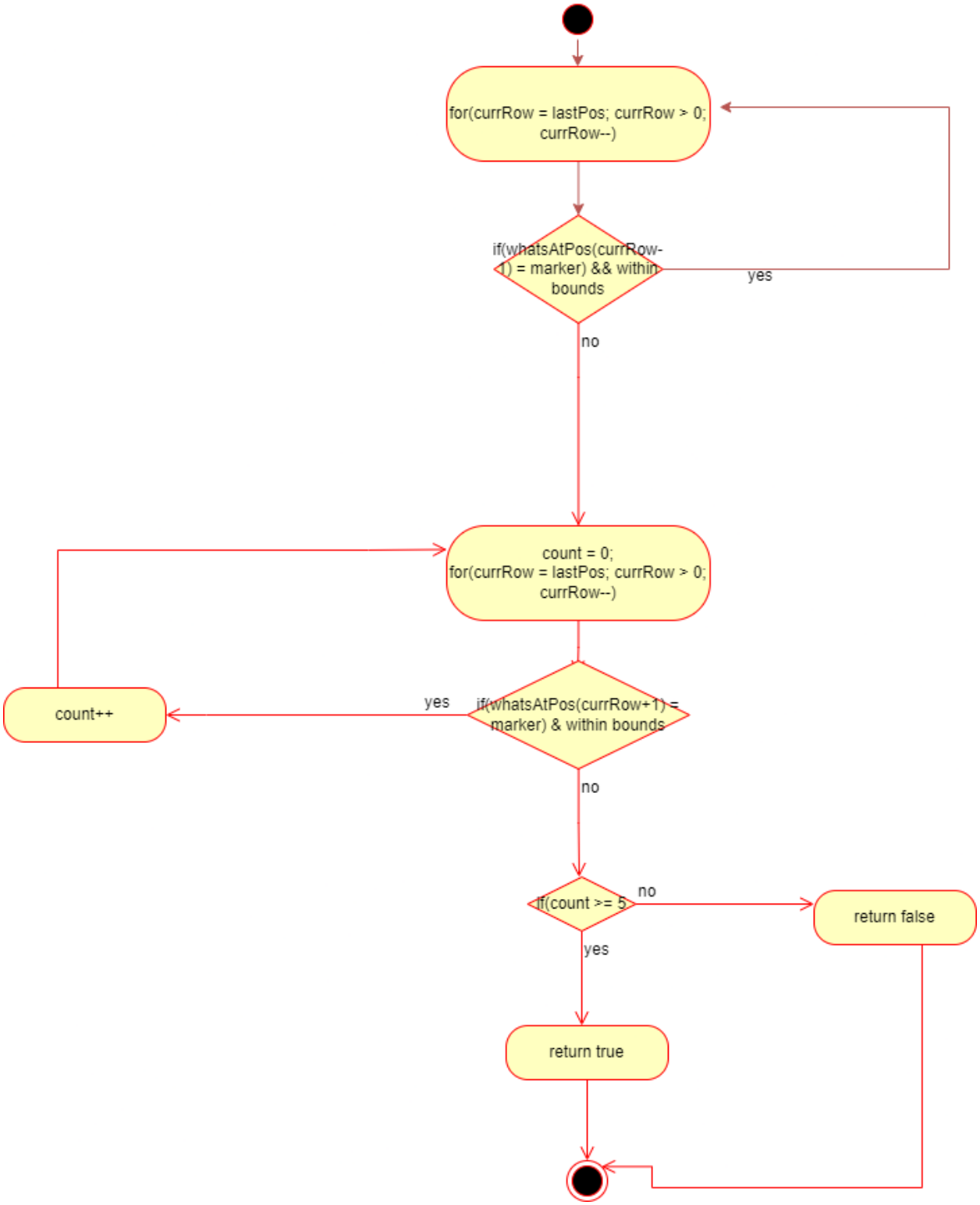
public boolean checkForDraw():



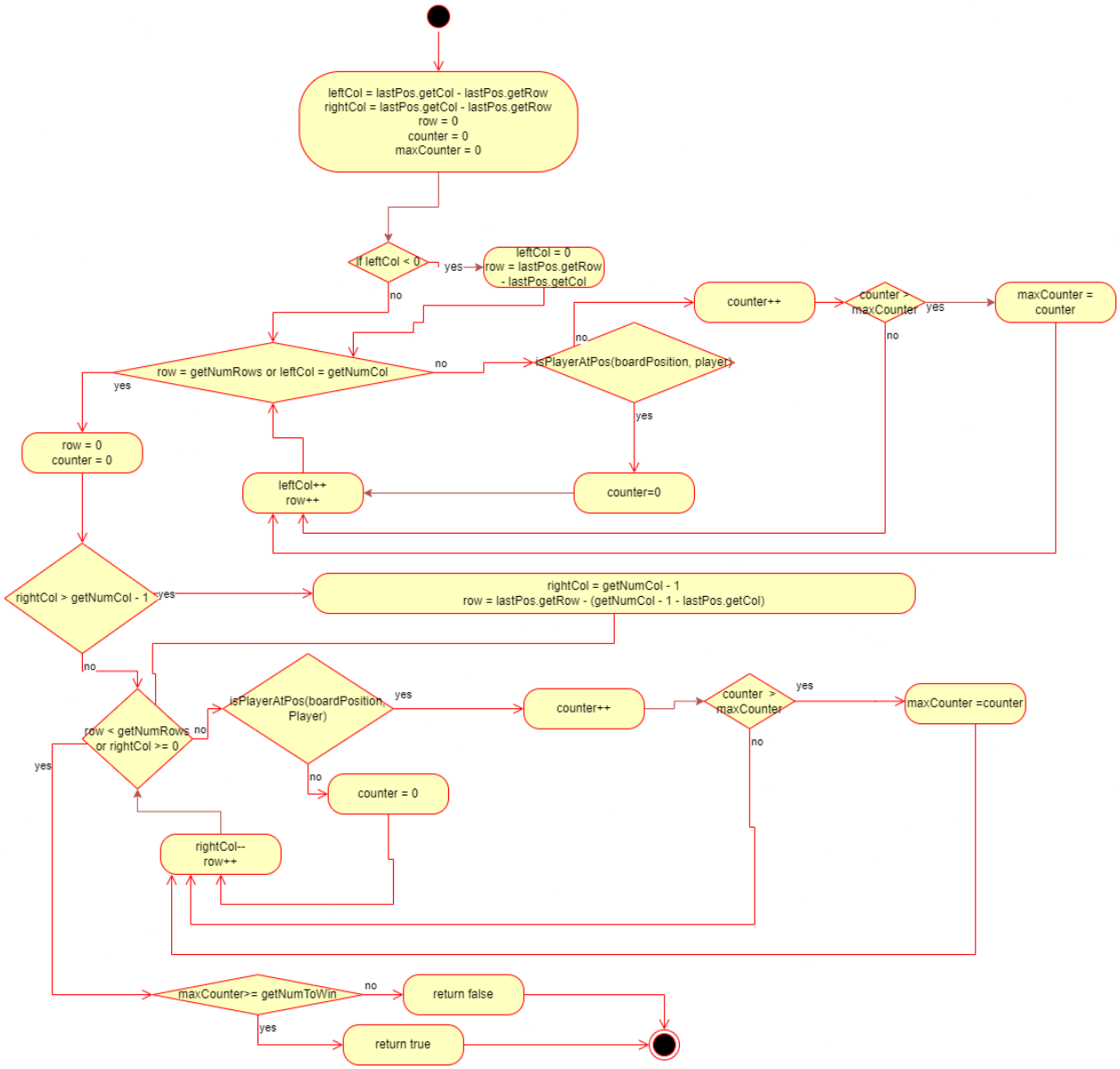
public boolean checkHorizontalWin(BoardPosition lastPos, char player):



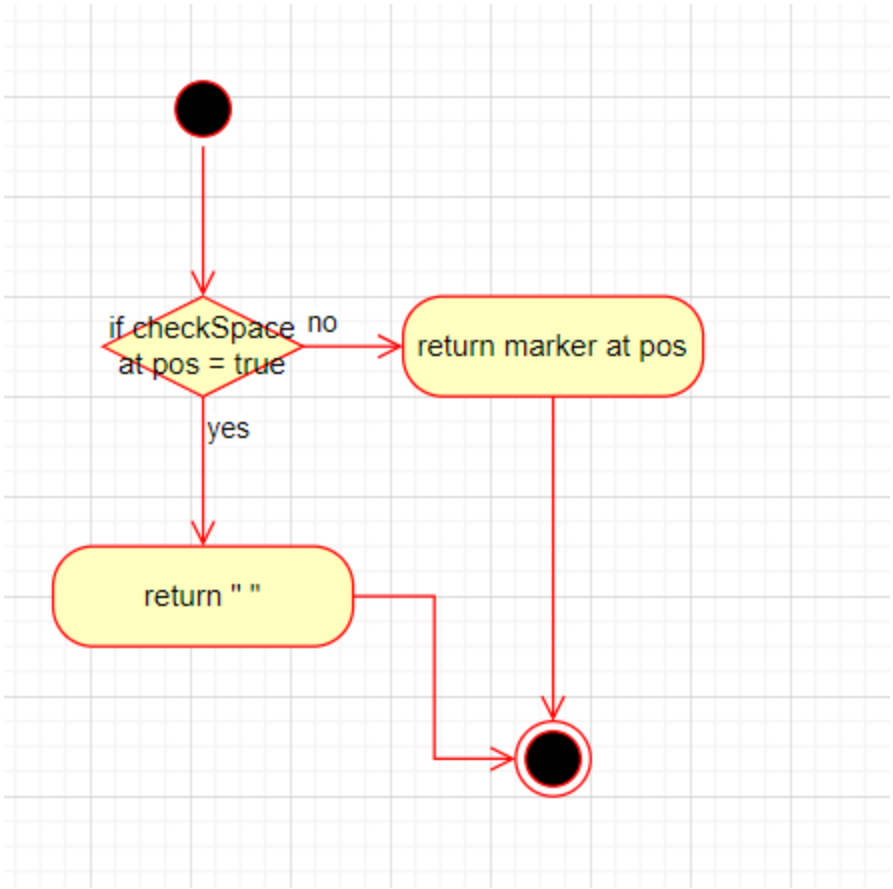
public boolean checkVerticalWin(BoardPosition lastPos, char player):



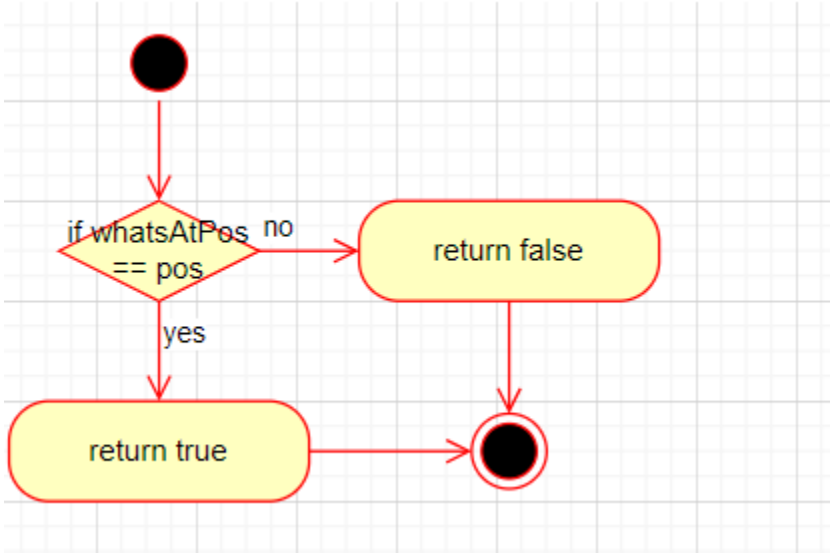
public boolean checkDiagonalWin(BoardPosition lastPos, char player):



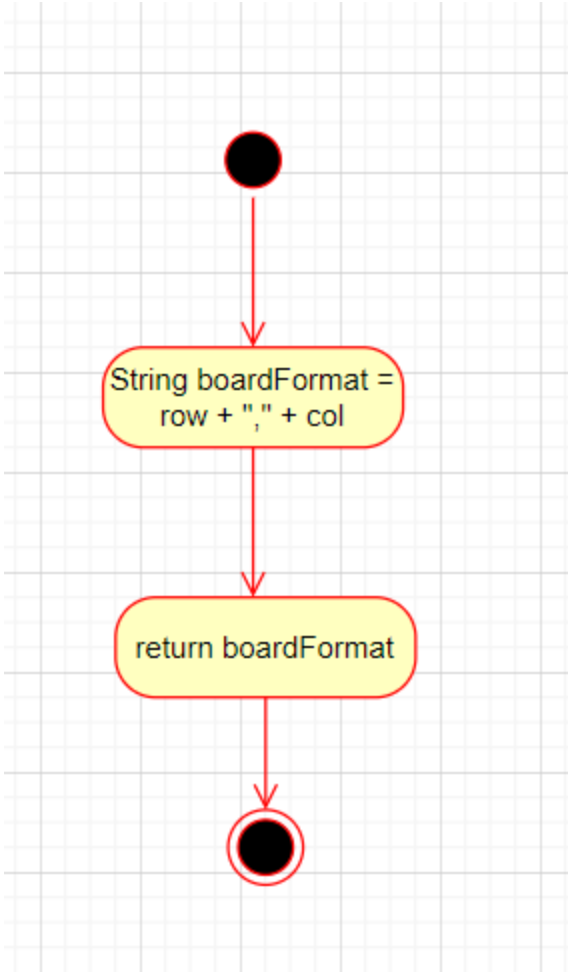
public char whatsAtPos(BoardPosition pos):



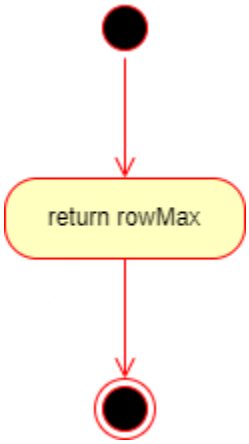
public boolean isPlayerAtPos(BoardPosition pos, char player):



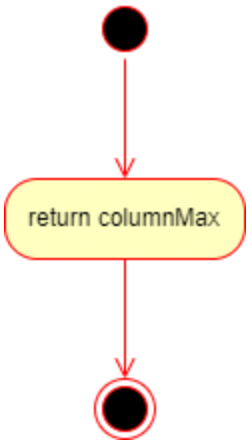
toString():



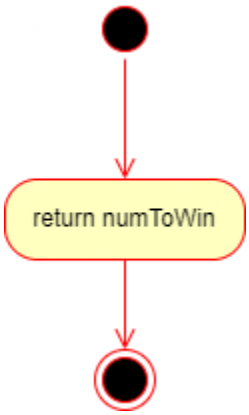
getNumRows():



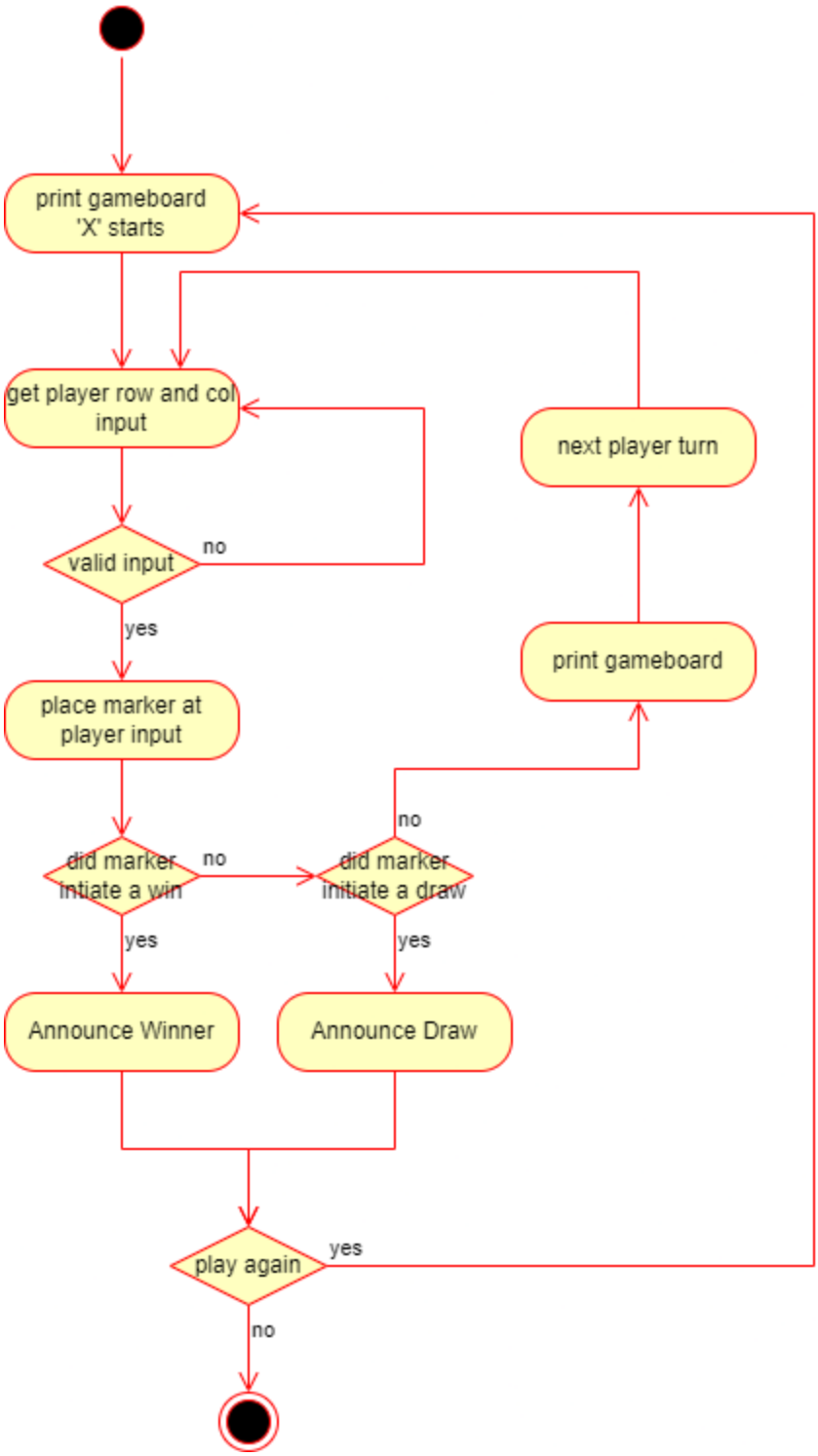
getNumCol();



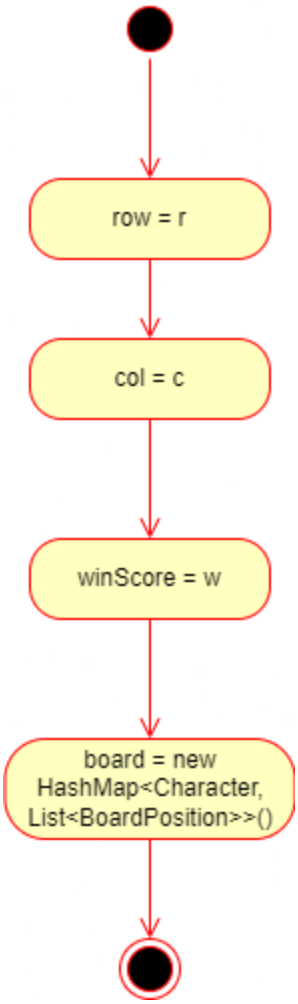
getNumToWin():



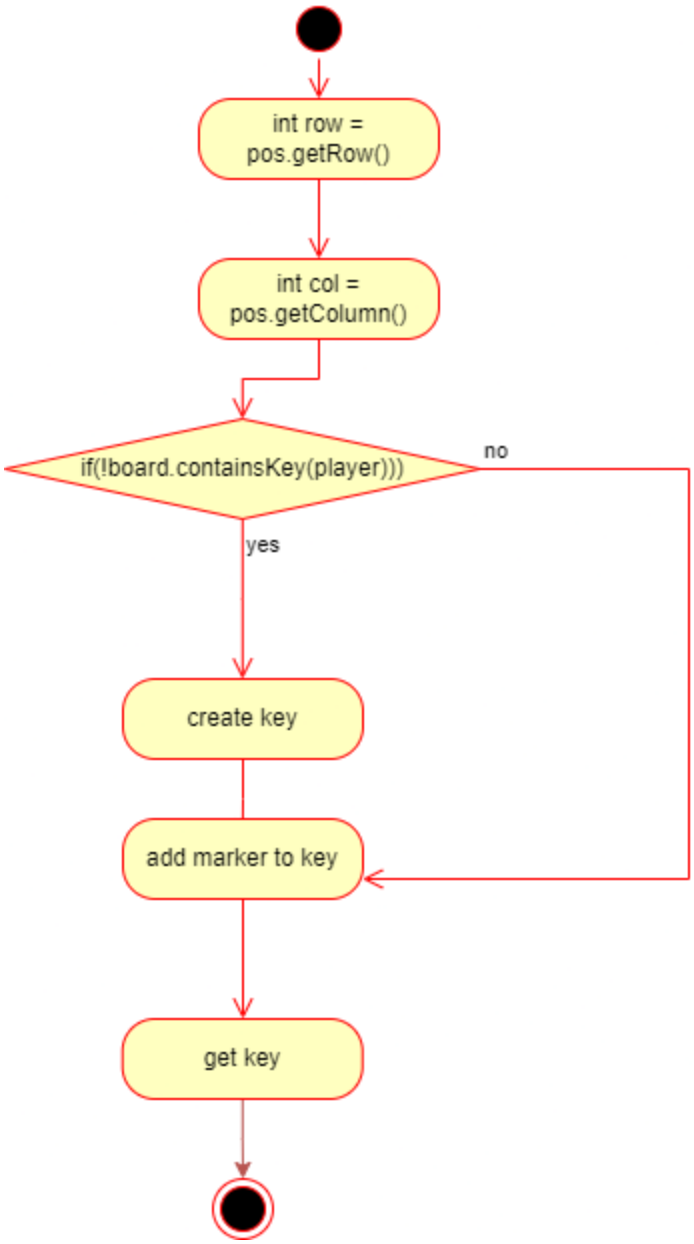
main():



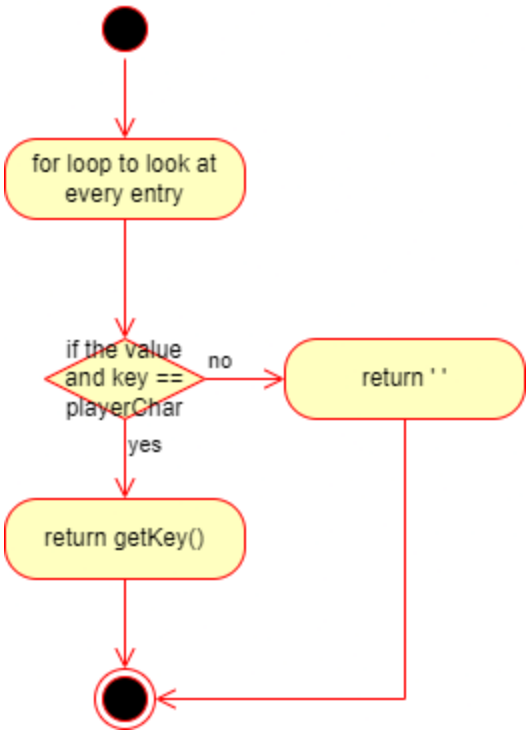
GameBoardMem(int r, int c, int w):



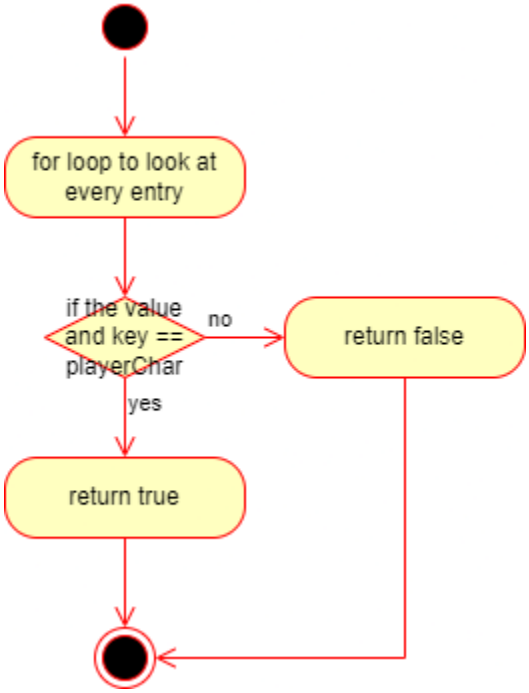
placeMarker(BoardPosition pos, char Player):



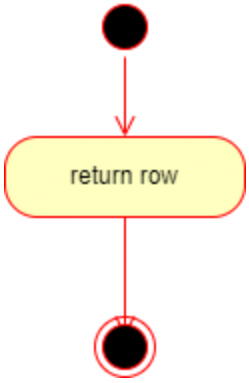
whatsAtPos(BoardPosition pos):



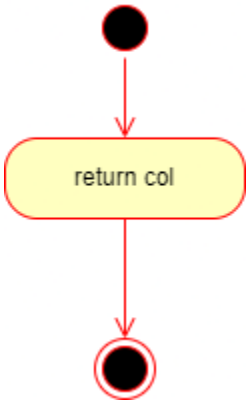
@Override isPlayerPos(BoardPosition pos, char playerChar):



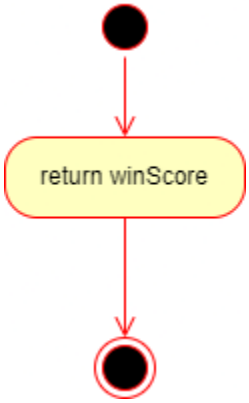
getNumRows():



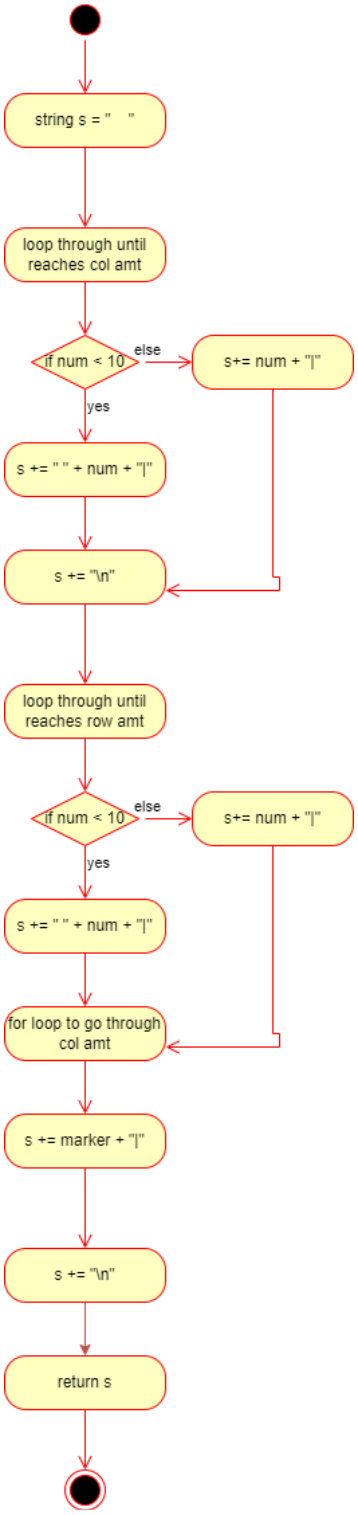
getNumColumns();



getNumToWin():



toString():



How To Run:

Once inside the Src directory, type “make” to compile code; then “run” to run the code; once the code has successfully finished, type “clean” to removes your compiled (.class) files.

Test Cases

Details in Project 4.

<u>Constructor:</u>

<table>
<tr>
<td>Input:
row = 3
col = 3
numToWin = 3</td>
<td>Output:
State:
<table>
<tr><td></td><td>0</td><td>1</td><td>2</td></tr>
<tr><td>0</td><td></td><td></td><td></td></tr>
<tr><td>1</td><td></td><td></td><td></td></tr>
<tr><td>2</td><td></td><td></td><td></td></tr>
</table>
getNumRows = 3
getNumColumns = 3
getNumToWin = 3</td>
<td>Reason:
This test is unique and distinct because it tests the minimum amount of board requirements needed for a table
Function name:
testGameBoard_min_input</td>
</tr>
<tr>
<td>input:
row = 5
col = 5
numToWin = 5</td>
<td>Output:
<table>
<tr><td></td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr>
<tr><td>0</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>1</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>2</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>3</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>4</td><td></td><td></td><td></td><td></td><td></td></tr>
</table>
getNumRows = 5
getNumColumns = 5
getNumToWin = 5</td>
<td>Reason:
This test is unique and distinct because it tests bounds of valid values thats not the min or max
Function name:
testGameBoard_valid</td>
</tr>
</table>

<table>
<tr>
<td>input:
row = 100
col = 100
numToWin = 25</td>
<td>Output:
<table><tr><td></td><td>0</td><td>1</td><td>2</td><td>…</td><td>99</td></tr><tr><td>0</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>…</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>99</td><td></td><td></td><td></td><td></td><td></td></tr></table>getNumRows = 100
getNumColumns = 100
getNumToWin = 25</td>
<td>Reason:
This test is unique and distinct because it tests the maximum amount of board requirements needed for a table
Function name:
testGameBoard_max_input</td>
</tr>
</table>

<u>checkSpace:</u>

<table>
<tr>
<td>Input:
State: 3 to win
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>0</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td></td></tr><tr><td>2</td><td></td><td></td><td></td></tr></table>pos.getRow = 0
pos.getCol = 2</td>
<td>Output:
checkspace = true

state of the board is unchanged</td>
<td>Reason:
This test is unique and distinct because it is checking a space on an unpopulated board
Function name:
testCheckSpace_empty</td>
</tr>
<tr>
<td>Input:
State: 3 to win
<table><tr><td></td><td>0</td><td>1</td><td>2</td></tr><tr><td>0</td><td>X</td><td></td><td>O</td></tr><tr><td>1</td><td>X</td><td>X</td><td></td></tr><tr><td>2</td><td>O</td><td>O</td><td></td></tr></table>pos.getRow = 2
pos.getCol = 1</td>
<td>Output:
checkSpace = false

State: unchanged</td>
<td>Reason:
This space is unique because it is in a spot where a marker is already populated in
Function:
testCheckSpace_populated</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 3 to win

	0	1	2
0	X	O	X
1	O	X	O
2	X	X	O

pos.getRow = 2
pos.getCol = 1

</td>
<td>

Output:
checkSpace = false

State: unchanged

</td>
<td>

Reason:
This space is unique because the board is already full
Function:
testCheckSpace_full

</td>
</tr>
</table>

<u>checkHorizontalWin:</u>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2	X	X	X	X	
3	O	O	O	X	O
4					

pos.getRow = 2;
pos.getCol = 2;
pos = ‘X’

</td>
<td>

Output:
checkHorizontalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed in the middle of the string of 4 consecutive X’s as opposed to on the end, so the function needs to count X’s on the right and left

Function:
testCheckHorizontalWin_
win_last_marker_middle

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2	X	X	X	X	
3	O	O	O	X	O
4					

pos.getRow = 2;
pos.getCol = 3;
pos = ‘X’

</td>
<td>

Output:
checkHorizontalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the end of the string of 4 consecutive X’s, so the function needs to count X’s on the left

Function:
testCheckHorizontalWin_
win_last_marker_end

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2	X	X	X		
3	O	O	O	X	O
4					

pos.getRow = 3;
pos.getCol = 3;
pos = ‘X’

</td>
<td>

Output:
checkHorizontalWin = False
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at a spot that would not give it a win, thus resulting in a false value
Function:
testCheckHorizontalWin_last_marker_false

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2	X	X	X	X	
3	O	O	O	X	O
4					

pos.getRow = 2;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkHorizontalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the start of the string of 4 consecutive X’s, so the function needs to count X’s on the right

Function:
testCheckHorizontalWin_start

</td>
</tr>
</table>

<u>checkVerticalWin:</u>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O			
1	X	O			
2	X	O			
3	X				
4					

pos.getRow = 3;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last marker was at the end of the string of 4 consecutive X’s, so the function needs to count X’s from the bottom to the top.
Function:
testCheckVerticalWin_end

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O			
1	X	O			
2	X	O			
3	X				
4					

pos.getRow = 0;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last marker was at the start of the string of 4 consecutive X’s, so the function needs to count X’s from the top to the bottom.
Function:
testCheckVerticalWin_start

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O			
1	X	O			
2	X	O			
3	X				
4					

pos.getRow = 2;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last marker was at the middle of the string of 4 consecutive X’s, so the function needs to count X’s from both the top to the bottom and bottom to top.
Function:
testCheckVerticalWin_mid

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O			
1	X	O			
2		O			
3	X				
4	X				

pos.getRow = 4;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = false
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last marker was at the end of the string but also does not out put 4 consecutive markers for the string so it would result in a false
Function:
testCheckVerticalWin_space

</td>
</tr>
</table>

<u>CheckDiagonalWin:</u>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2			X		
3	O	O	O	X	
4			O		X

pos.getRow = 1;
pos.getCol = 1;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the start of the string of 4 diagonal X’s, so the function needs to count from the last placed marker, then going down and to the right

Function:
testCheckDiagonalWin_start

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2			X		
3	O	O	O	X	
4			O		X

pos.getRow = 4;
pos.getCol = 4;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the end of the string of 4 diagonal X’s, so the function needs to count from the last placed marker, then going up and to the left

Function:
testCheckDiagonalWin_end

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2			X		
3	O	O	O	X	
4			O		X

pos.getRow = 2;
pos.getCol = 2;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the middle of the string of 4 diagonal X’s, so the function needs to count from the last placed marker, then both going down and to the right and up and to the left

Function:
testCheckDiagonalWin_middle

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1				X	
2			X		
3		X		O	O
4	X			O	O

pos.getRow = 1;
pos.getCol = 3;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the start of the string of 4 diagonal X’s, so the function needs to count from the last placed marker, then going down and to the left

Function:
testCheckDiagonalWin_start_opposite

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1				X	
2			X		
3		X		O	O
4	X			O	O

pos.getRow = 4;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the end of the string of 4 diagonal X’s, so the function needs to count from the last placed marker, then going up and to the right

Function:
testCheckDiagonalWin_end_opposite

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1				X	
2			X		
3		X		O	O
4	X			O	O

pos.getRow = 2;
pos.getCol = 2;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last X was placed at the middle of the string of 4 diagonal X’s, so the function needs to count from the last placed marker, then both going down and to the left and up and to the right

Function:
testCheckDiagonalWin_middle_opposite

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					X
1				X	
2					
3		X		O	O
4	X			O	O

pos.getRow = 0;
pos.getCol = 4;
pos = ‘X’

</td>
<td>

Output:
checkVerticalWin = false
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the last marker was at the end of the string but also does not out put 4 consecutive markers for the string so it would result in a false

Function:
testCheckDiagonalWin_false

</td>
</tr>
</table>

<u>checkForDraw:</u>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2					
3					
4					

</td>
<td>

Output:
checkForDraw = false
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the board is empty, so the function only would result in a false value
Function name:
testCheckForDraw_empty

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O	X	O	X
1	O	X	O	X	O
2	O	X	X	O	X
3	X	O	X	O	X
4	O	X	O	X	O

</td>
<td>

Output:
checkForDraw = true
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the board is full and there is no win condition through either marker, so the function only would result in a true value
Function name:
testCheckForDraw_full

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0		O	X	O	X
1	O	X	O	X	O
2	O	X	X	O	X
3	X	O	X	O	X
4	O	X	O	X	O

pos.getRow = 0;
pos.getCol = 0;
pos = ‘X’

</td>
<td>

Output:
checkForDraw = false
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the board’s first position is empty, so the function would result in a false value.
Function name:
testCheckForDraw_first_empty

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O	X	O	X
1	O	X	O	X	O
2	O	X	X	O	X
3	X	O	X	O	X
4	O	X	O	X	

pos.getRow = 4;
pos.getCol = 4;
pos = ‘O’

</td>
<td>

Output:
checkForDraw = false
state is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because the board’s last position is empty, so the function would result in a false value.
Function name:
testCheckForDraw_last_empty

</td>
</tr>
</table>

<u>whatsAtPos:</u>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2					
3					
4					

pos.getRow = 1
pos.getCol = 1

</td>
<td>

Output:
wharsAtPos = ‘ ‘
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the board is empty, which will make the function return a space.
Function:
testWhatsAtPos_empty

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X				
1	O				
2	X				
3	O				
4	X				

pos.getRow = 0
pos.getCol = 1

</td>
<td>

Output:
wharsAtPos = ‘ ‘
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the space is next to a filled
column, which will make the
function return a space.
Function:
testWhatsAtPos_adjacent_spac
e

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X	O	X	O	X
1	O	X	O	X	O
2	O	X	X	O	X
3	X	O	X	O	X
4	O	X	O	X	

pos.getRow = 4;
pos.getCol = 4;

</td>
<td>

Output:
wharsAtPos = ‘ ‘
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the board is filled except one,
which will make the function
return a space.
Function:
testWhatsAtPos_single_space

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2					
3					
4					

pos.getRow = 1
pos.getCol = 1

</td>
<td>

Output:
wharsAtPos = ‘X ‘
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the board is empty except for
one, which will make the
function return the marker.
Function:
testWhatsAtPos_single_marker

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0	X				
1	O		X		O
2					
3	O			X	
4					

pos.getRow = 1
pos.getCol = 0

</td>
<td>

Output:
wharsAtPos = ‘O ‘
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the space on the board is filled,
which will make the function
return the marker.
Function:
testWhatsAtPos_filled

</td>
</tr>
</table>

<u>isPlayerAtPos:</u>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1					
2					
3					
4					

pos.getRow = 1
pos.getCol = 1
pos = ‘X’

</td>
<td>

Output:
isPlayerAtPos = false
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the board is empty, which will
test to see if a single marker is in
an empty board.
Function:
testIsPlayerAtPos_empty

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2					
3					
4					

pos.getRow = 1
pos.getCol = 1
pos = ‘X’

</td>
<td>

Output:
isPlayerAtPos = true
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the board will test to see if a
single marker is in position
Function:
testIsPlayerAtPos_single_marker

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2				O	
3					
4					

pos.getRow = 2
pos.getCol = 3
pos = ‘O’

</td>
<td>

Output:
isPlayerAtPos = true
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the board will test to see if the
function can differentiate
between two different markers
Function:
testIsPlayerAtPos_multiple_mar
kers

</td>
</tr>
</table>

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3	4
0					
1		X			
2				O	
3					
4					

pos.getRow = 2
pos.getCol = 3
pos = ‘X’

</td>
<td>

Output:
isPlayerAtPos = false
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the pos is different and marker is already taken by a different marker
Function:
testIsPlayerAtPos_different_marker

</td>
</tr>
<tr>
<td>

Input:
State: 5 to win

	0	1	2	3	4
0	A	S	D	F	G
1	S	D	F	G	A
2	D	F	G	A	S
3	F	G	A	S	D
4	A	S	D	F	G

pos.getRow = 3
pos.getCol = 3
pos = ‘S’

</td>
<td>

Output:
isPlayerAtPos = true
state is unchanged

</td>
<td>

Reason:
This test case is unique
and distinct because
the pos is filled and the board is filled
Function:
testIsPlayerAtPos_filled

</td>
</tr>
</table>

placeMarker:

Input:
State: 4 to win

	0	1	2	3
0				
1				
2		X		
3	O			

pos.getRow = 1;
pos.getCol = 2;
p = ‘A’

Output:
State:

	0	1	2	3
0				
1			A	
2		X		
3	O			

Reason:
This test case is unique and distinct because I am placing a marker representing a player who has not been placed on this board before.

Function:
testPlaceMarker_col_not_empty

Input:
State: 4 to win

	0	1	2	3
0				
1				
2				
3				

pos.getRow = 1;
pos.getCol = 2;
p = ‘X’

Output:
State:

	0	1	2	3
0				
1		X		
2				
3				

Reason:
This test case is unique and distinct because I am placing a marker representing a player when the board is empty.

Function:
testPlaceMarker_col_empty

Input:
State: 4 to win

	0	1	2	3
0	X	O	X	O
1	O	X		X
2	X	O	X	O
3	O	X	X	O

pos.getRow = 1;
pos.getCol = 2;
p = ‘O’

Output:
State:

	0	1	2	3
0	X	O	X	O
1	O	X	O	X
2	X	O	X	O
3	O	X	X	O

Reason:
This test case is unique and distinct because I am placing a marker representing a player when the board is filled except one space.

Function:
testPlaceMarker_single_space

<table>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3
0	X	O		
1		X		X
2		O		
3			O	

pos.getRow = 1;
pos.getCol = 2;
p = ‘H’

</td>
<td>

Output:
State is unchanged

</td>
<td>

Reason:
This test case is unique and distinct because I am placing a marker that is not supposed to be in the gameboard.

Function:
testPlaceMarker_invalid_marker

</td>
</tr>
<tr>
<td>

Input:
State: 4 to win

	0	1	2	3
0				
1				
2		X		
3	O			

pos.getRow = 1;
pos.getCol = 2;
p = ‘X’

</td>
<td>

Output:
State:

	0	1	2	3
0				
1			X	
2		X		
3	O			

</td>
<td>

Reason:
This test case is unique and distinct because I am placing a marker representing a player who has been placed on this board before.

Function:
testPlaceMarker_same_marker

</td>
</tr>
</table>